

ALFA PDF READER

Plano de Melhorias

Versão atual: 1.2.0 Stable

Data: Julho 2026

Desenvolvido por Alex A. Alves

VISUAL | PERFORMANCE | QUALIDADE

Melhorias Visuais

1. Header Dinâmico com Identidade

- * Header retrátil com logo + nome do PDF ativo + botão de abrir arquivo
- * Animação de collapse ao scrollar (esconde durante leitura, mostra ao voltar ao topo)
- * Barra de busca textual (Ctrl+F) integrada no header
- * Indicador de página e progresso no header quando colapsado

2. Sidebar de Thumbnails (Navegação Visual)

- * Painel retrátil à esquerda com miniaturas das páginas via PDF.js
- * Cada thumbnail com número da página e destaque na página ativa (glow vermelho)
- * Transição lateral suave (translateX + width animado)
- * Toggle button no canto esquerdo da toolbar
- * Implementação: canvas renderizado em offscreen com escala reduzida

3. Transições e Micro-animações

- * FadeIn + SlideUp ao mudar de página via botões de navegação
- * Skeleton shimmer enquanto PDF carrega (substitui loading spinner fixo)
- * Ripple effect nos botões da toolbar ao clicar
- * AnimatePresence nas abas (opacity + transform ao fechar/abrir)
- * Transição suave de cores no hover dos botões

4. Efeito de Papel Realista nas Páginas

- * Page-curl shadow: sombra 3D com gradiente no canto inferior direito
- * Paper texture sutil via SVG filter nas páginas A4
- * Sombra projetada multi-camada com filter: drop-shadow()
- * Glow sutil entre páginas (box-shadow: 0 0 30px rgba(228,0,43,0.05))

5. Barra de Progresso de Leitura

- * Thin progress bar no topo da janela (estilo Medium/Youtube)
- * Scrollbar customizada com gradiente dinâmico
- * Indicador de progresso percentual ao lado do nome do arquivo

6. Componentes Adicionais

- * FAB "Voltar ao Topo" com animação de entrada ao scrollar
- * Toggle de tema Dark / Midnight no header
- * Print Dialog com preview visual das páginas selecionadas
- * Indicador visual de páginas selecionadas no preview de impressão

Qualidade de Impressão - Planejamento

Propõe-se 4 níveis de qualidade de impressão, controlando resolução, compressão e processamento das páginas antes de enviar à impressora.

MÁXIMA (Alta)

Para documentos finais, apresentações, materiais para clientes

Resolução: 300 DPI | Sem compressão de imagem | Sem redução de cor | PDF.js rendering em escala 2.0 (devicePixelRatio máximo) | Criação de bitmap com ImageBitmap para alta fidelidade

MÉDIA

Para uso interno, documentos do dia a dia

Resolução: 200 DPI | Compressão JPEG leve (qualidade 85%) | Preserva cores | PDF.js rendering em escala 1.5 | Balanceia qualidade e performance

NORMAL (Padrão)

Configuração padrão, bom equilíbrio

Resolução: 150 DPI | Compressão JPEG moderada (qualidade 70%) | Preserva cores | PDF.js rendering em escala 1.2 | Adequado para a maioria dos casos

RASCUNHO (Econômico)

Para revisão rápida, economia de tinta/papel

Resolução: 100 DPI | Compressão JPEG alta (qualidade 50%) | Converte para grayscale automaticamente | PDF.js rendering em escala 0.8 | Remove background images | Modo toner saving

Implementação Técnica

A qualidade deve ser controlada no pipeline de impressão (main/index.ts) com os seguintes parâmetros:

```
interface PrintQualityOptions {  
  quality: "maxima" | "media" | "normal" | "rascunho"  
  dpi: number // 300 | 200 | 150 | 100  
  jpegQuality: number // 100 | 85 | 70 | 50  
  scale: number // 2.0 | 1.5 | 1.2 | 0.8  
  grayscale: boolean // false | false | false | true  
  removeBg?: boolean // false | false | false | true  
}
```

No PrintDialog, adicionar um RadioGroup "Qualidade de Impressão" com os 4 níveis, cada um com tooltip explicativo e ícone representativo.

Velocidade de Abertura - Otimizações

Diagnóstico atual: O PDF é carregado via FileReader => ArrayBuffer => Blob => URL.createObjectURL => PDF.js getDocument(). Cada etapa é sequencial. Para documentos grandes (+100MB), o processo pode levar vários segundos.

1. Lazy Loading com Streaming (Alto Impacto)

Em vez de ler o arquivo inteiro na RAM antes de processar, usar o cabeçalho do PDF para iniciar o parse enquanto o resto carrega. PDF.js suporta fetch progressivo com Range Requests.

```
// Atual: carrega tudo na memória
const arrayBuffer = await file.arrayBuffer()
const url = URL.createObjectURL(new Blob([arrayBuffer]))

// Novo: streaming direto do File
const url = URL.createObjectURL(file) // File já é um Blob!
// PDF.js faz fetch progressivo automaticamente
```

- * **Eliminar conversão File => ArrayBuffer => Blob (redundante)**
- * O File já é um Blob - pode gerar URL direto
- * PDF.js lê o header primeiro e já exibe a página 1 enquanto o resto carrega em background
- * Impacto estimado: 40-60% mais rápido na primeira página visível

2. Cache de Documentos Abertos (Médio Impacto)

Manter documentos PDF.js carregados em cache para reabertura instantânea de abas recentes.

```
const pdfCache = new Map<string, PDFDocumentProxy>()

async function getCachedDocument(url: string, options?: any) {
  const key = url
  if (pdfCache.has(key)) return pdfCache.get(key)!
  const doc = await pdfjsLib.getDocument({ url, ...options }).promise
  pdfCache.set(key, doc)
  return doc
}

// Limpar cache ao fechar aba
function closeTab(id: string) {
  pdfCache.delete(getUrlForTab(id))
  URL.revokeObjectURL(tab.url) // FIX: memory leak atual
}
```

- * Cache LRU para evitar acúmulo infinito de memória
- * Revogar URLs antigas (URL.revokeObjectURL) - atualmente é um memory leak
- * Reabrir aba recente = instantâneo (sem reprocessar PDF.js)

3. Web Workers Dedicados (Médio Impacto)

PDF.js já usa Web Worker por padrão, mas a inicialização do worker tem custo. Pool de workers pré-inicializados.

```
const workerPool = new Array(2).fill(null).map(() => {
  const worker = new Worker("pdf.worker.min.mjs")
  return worker
})

// Rotacionar workers entre abas para paralelismo real
const worker = workerPool[nextWorker % workerPool.length]
```

- * Pool de 2-4 workers pré-criados na inicialização
- * Documentos grandes processados em worker separado sem travar UI
- * Priorizar worker para a aba ativa, segundo worker para pré-carregar próxima aba

Velocidade de Abertura - Continuação

4. Pré-carregamento da Página Seguinte (Baixo Impacto, Alta Percepção)

Enquanto o usuário lê a página N, renderizar a página N+1 em background. A navegação fica instantânea.

```
// No PdfViewer, ao carregar página N, inicia render de N+1
useEffect(() => {
  if (currentPage < totalPages) {
    const nextPage = document.querySelector(
      `[data-page-number="${currentPage + 1}"]`
    )
    // react-pdf já faz isso internamente,
    // mas podemos forçar prioridade
  }
}, [currentPage])
```

5. Compressão e Otimização de Imagens

PDFs escaneados (imagens pesadas) podem ser otimizados durante a abertura:

- * Detectar páginas como imagem vs vetor
- * Redimensionar imagens para a resolução da tela (nunca maior que o viewport)
- * Usar `createImageBitmap()` com `option resizeQuality` para `downscale` rápido
- * Descartar imagens abaixo de 8x8 px (ruído de digitalização)

6. Virtualização da Lista de Páginas (Escalabilidade)

Para documentos com 100+ páginas, renderizar apenas as páginas visíveis no viewport (viewport virtualization) em vez de todas.

```
// Em vez de renderizar 300 páginas de uma vez:
const visiblePages = getVisibleRange(containerRef, numPages)

{Array.from({ length: numPages }, (_, i) => {
  if (i + 1 < visiblePages.start || i + 1 > visiblePages.end)
    return <div key={i} style={{ height: estimatedPageHeight }} />
  return <Page pageNumber={i + 1} />
})}
```

- * Determinar altura estimada por página com base na primeira página renderizada
- * Manter 5 páginas de buffer acima/abaixo do viewport
- * Reciclar canvases: reutilizar elementos canvas em vez de criar/remover DOM
- * Redução de memória: de centenas de canvases para ~10-15 simultâneos

Priorização e Esforço Estimado

Estimativa de Esforço

Legenda: [VIS] Impacto Visual | [PERF] Performance | [ESF] Esforço de Implementacao

Melhoria | Tipo | Impacto | Esforço | Prioridade

Eliminar conversão File=>ArrayBuffer=>Blob	Perf	Alto	2h	P0
Cache de documentos PDF.js (LRU)	Perf	Alto	4h	P0
Qualidade de impressão (4 níveis)	Feature	Alto	8h	P0
Sidebar de thumbnails	Visual	Alto	16h	P1
Virtualização de páginas (100+)	Perf	Alto	12h	P1
Header dinâmico retrátil	Visual	Médio	6h	P1
Pré-carregamento página seguinte	Perf	Médio	3h	P1
Transições e micro-animações	Visual	Médio	8h	P2
Efeito papel realista	Visual	Médio	6h	P2
Barra de progresso de leitura	Visual	Baixo	3h	P2
FAB Voltar ao Topo	Visual	Baixo	2h	P3
Pool de Web Workers	Perf	Médio	8h	P3
Toggle tema Dark/Midnight	Visual	Baixo	2h	P3
Print preview visual	Visual	Médio	10h	P3

Resumo

As melhorias foram divididas em 3 frentes principais:

- * Performance (P0): streaming direto do File, cache LRU, virtualização - impacto imediato na abertura de documentos
- * Qualidade de Impressão (P0): 4 níveis com controle de DPI, compressão e escala - diferencia o produto profissionalmente
- * Visual (P1-P3): thumbnails, header dinâmico, transições - elevam a percepção de qualidade do produto

Tempo total estimado: ~90 horas de desenvolvimento para implementação completa.

Versão mínima viável (P0 + P1): ~45 horas - entrega performance + qualidade de impressão + thumbnail + header.